Manual Testing Documentation

Manual Functional Testing

Tab 1: Import Data Tab

Test case 1: browse and upload files

Description: Verify that the file browsing and upload functionality works correctly.

Steps:

1. Click the ‘Browse Files’ button
2. Select and upload a .csv or .otb file

Expected Results:

- Expected UI behaviour:
    - File dialog opens up displaying supported file types (.csv and .otb).
    - After uploading a valid file, the filename and data appears in the interface. The file should also appear under ‘Recent Files’.

- Expected backend behaviour:
    - The system reads the selected file and validates its format.

Test case 2: Invalid file handling

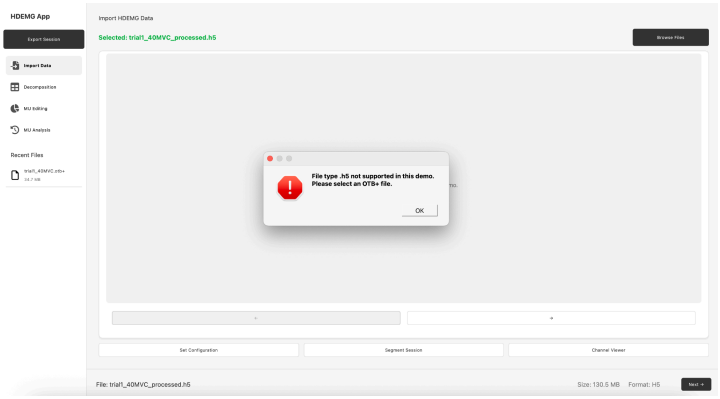
Description: Verify that the file uploading functionality handles errors correctly.

Steps:

1. Click the ‘Browse Files’ button
2. Try to upload an unsupported file format

Expected Results:

- Expected UI behaviour:
    - File dialog opens up displaying data files.
    - Uploading an unsupported file triggers an error popup.



- Expected backend behaviour:
    - The system reads the selected file and validates its format.
    - Errors are logged if the file type or content is invalid.

Test 3: Set configuration splitter and input configuration

Description: Verify splitter and input configurations are correctly detected.

Steps:

1. Click the ‘Set Configuration’ button
2. Check all splitter and input configurations
3. Click done to apply configuration
4. Verify changes in the import data tab

Expected Results:

- Expected UI behaviour:
    - Configuration dialog opens
    - All quattrocento visualisations should turn green (indicating successful configuration)
    - After clicking done, the configuration dialog closes. Main preview plot updates to reflect new channel configuration.
- Expected backend behaviour:
    - Qt signals emitted to update dependent components

        - Preview plot refreshes with new configurations
        - Channel viewer updates to show/hide disabled channels
        - Electrode navigation recalculates grid indices

Test 4: Segment session button availability

Description: Verify availability of the ‘Segment Session’ button

Steps:

1. Load a valid EMG file via the ‘Browse File’ button
2. Click on the ‘Segment Session’ button

Expected Results:

- Expected UI behaviour:
    - The ‘Segment Session’ button is disabled if no file has been uploaded.
    - The ‘Segment Session’ button should become available after uploading a valid file.
- Expected backend behaviour:
    - No segmentation processing is triggered when attempting to click disabled button.
    - Button state is controlled by file loading status.

Test 5: Segment session functionality - Concatenate

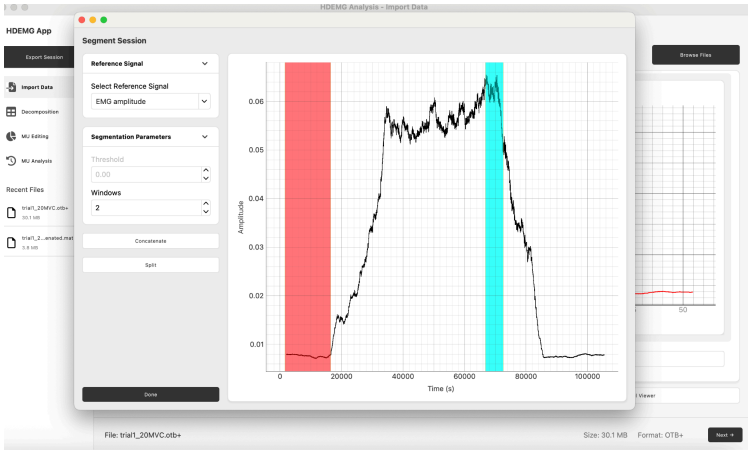
Description: Verify the correctness of ‘concatenate’ functionality for combining multiple EMG data segments into a single continuous session.

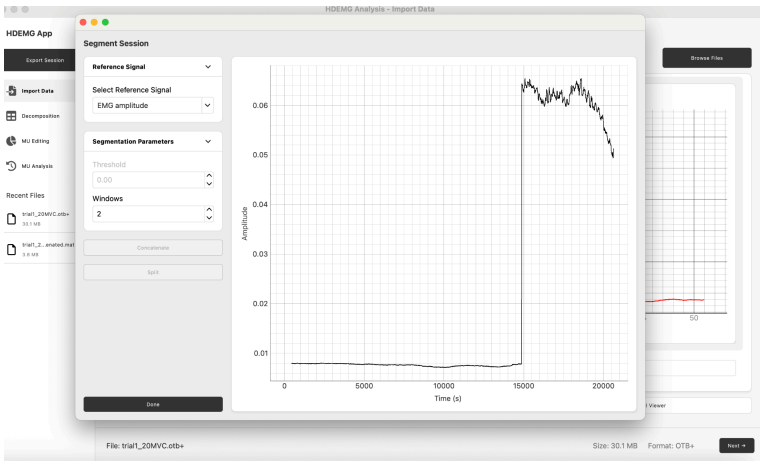
Steps:

1. Click on the ‘Segment Session’ button
2. Reference signal should be set to ‘EMG Amplitude’
3. Set ‘windows’ in segmentation parameters to 2
4. Drag highlighted sections to desired concatenation width and position
5. Click the ‘concatenate’ button

Expected Results:

- Expected UI behaviour:
    - Once number of windows in segmentation parameters is set to 2, 2 highlighted section will appear
    - After dragging the highlighted sections to desired time location (image 1), the ‘concatenate’ button will be able to produce the concatenated data (image 2).





- Expected backend behaviour:
    - EMG amplitude reference signal is correctly calculated and applied.
    - Selected data segments are extracted without data loss.
    - Clicking ‘concatenate’ when windows is set to less than 1 will produce an error message in the backend.

Test 6: Segment session functionality - Split

Description: Verify the correctness of the ‘split’ functionality for dividing EMG data into multiple separate segments for individual analysis.

Steps:

1. Click on the ‘Segment Session’ button
2. Reference signal should be set to ‘EMG Amplitude’
3. Set ‘windows’ in segmentation parameters to 3
4. Drag highlighted sections to desired split position

5. Click the ‘split’ button

Expected Results:

- Expected UI behaviour:
    - After changing windows in the segmentation parameters to 3, 3 draggable sections will appear.
    - After dragging sections to desired time positions, the ‘split’ button will become clickable.
    - The segments that have been split will appear as .mat files under ‘Recent Files’ on the left sidebar.
- Expected backend behaviour:
    - Windows parameter properly configures the segmentation detection algorithm.
    - Each split segment receives a unique session identifier.

Test 6: Channel viewer

Description: Verify the channel viewer correctly displays individual EMG channels, allows for channel selection, navigation and provides accurate visualisation of EMG data signals.

Steps:

1. Click on the ‘Channel Viewer’ button.
2. Test navigation to next and previous channels by clicking the ‘<--’ and ‘-->’ buttons.
3. Toggle the number of signals to display.

Expected Results:

- Expected UI behaviour:
    - The channel viewer window opens once the ‘channel viewer’ button has been clicked.
    - Channel metadata updates immediately upon channel selection.
        - Selected channel highlighted in plot area
        - Channel number displayed
    - Smooth transitions when toggling parameters.
- Expected backend behaviour:
    - EMG signal data accessed via emg_obj.signal_dict
    - Signal plot updates triggered via Qt signal mechanism when navigation occurs

<u>Tab 2: Decomposition Tab</u>

Test 1: Decomposition Process

Description: Test the decomposition process lifecycle including starting, stopping, continuing and restarting EMG signal decomposition. This test validates the decomposition workflow and behaviour.

Steps:

1. Upload a valid data file in the ‘import data’ tab.
2. Click the ‘next’ button. The file will now be loaded into the decomposition tab.
3. Configure decomposition parameters:
    a. Select algorithm - FastICA
    b. Set parameter value to 20
4. Click the ‘start decomposition’ button.
5. Monitor the top information bar for the decomposition progress and status updates.
6. Click the ‘stop decomposition’. This will pause the decomposition progress in the frontend and backend.
7. Click the ‘continue decomposition’. This will continue the decomposition progress from where it left off.
8. Click the ‘restart decomposition’ button. This will restart the entire decomposition process and restore the data back to its original state.

Expected Results:

- Expected UI behavior:
    - Start decomposition
        - If no file is uploaded, the ‘start’ button will be a light grey colour and not be clickable.
        - Once a file has been uploaded, the ‘start’ button will become clickable.
        - The status bar will begin updating users with the decomposition progress (e.g. “Processing electrode 1/2”).
        - Signal preview plot shows original EMG data.
        - Pulse train plot updates as motor units are identified.
    - Stop decomposition
        - The ‘stop’ decomposition button will also become clickable once the ‘start’ button has been clicked.
        - Observe UI state changes and status updates.
    - Continue decomposition
        - ‘Continue’ button will not be visible until the ‘Stop’ button has been clicked.
        - ‘Start/Stop’ buttons will reappear once the ‘Continue’ button has been clicked.
        - Observe the top status bar as the progress will start updating again.
    - Restart decomposition
        - The ‘Restart’ button will be visible once the ‘Stop’ button has been clicked.
        - Clicking the ‘Restart’ button will restore the original data state.

- Expected backend behaviour:
    - Decomposition results stored in emg_obj.mu_dict and emg_obj.decomp_dict.
    - The backend pauses the decomposition when stop is clicked.
    - Once a decomposition process is continued, previous ui_params and algo_choice are restored and partial results are preserved and extended.

Tab 3: MU Editing Tab

Test 1: Adding, deleting and locking spikes

Description: test the manual spike editing functionality including adding new spikes, deleting existing spikes and locking spikes to prevent modification during filter updates.

Steps:

1. Load decomposed EMG data with identified motor units by clicking the ‘next’ button in the decomposition tab.
2. Select a motor unit from the MU checkbox list,
3. Adding spikes:
    a. Click the ‘Add Spikes’ button.
    b. Mark region for new spikes.
    c. Confirm spike addition
4. Deleting spikes:
    a. Click the ‘Delete Spike’ button
    b. Draw a selection rectangle around existing spikes to be removed.
    c. Confirm spike deletion.
5. Locking spikes:
    a. Click the ‘lock spikes’ button to preserve current spikes
    b. Attempt filter update to verify locked spikes remain unchanged.

Expected Results:

- Expected UI behaviour:
    - Add/Delete/Lock Spikes buttons become active when MU is selected.
    - Selection tool activates allowing rectangle drawing on spike train plot.
    - Real-time visual feedback during selection.
    - Spike train updates immediately after adding/deleting spikes.
    - Confirmation displaying success/failure.
- Expected backend behaviour:
    - add_spikes_button_pushed() creates SelectionTool for interactive selection.
    - delete_spikes_button_pushed() removes spikes within selection bounds.
    - Selection coordinates processed by process_selection().

Test 2: Update MU filters

Description: Test the motor unit filter update functionality.

Steps:

1. Ensure decomposed EMG data is loaded in from the decomposition tab.
2. Select a motor unit from the checkbox list.
3. Optionally modify spikes using the steps in test 1.
4. Click the ‘Update MU Filter’ button.
5. Monitor progress and SIL value changes.
6. Observe colour coded feedback.
7. Verify updated pulse train visualisation.

Expected Results:

- Expected UI behaviour:
    - ‘Update Filter’ button enabled when single MU selected.
    - Real-time progress updates in the status area.
    - SIL value updates in motor unit information display.
    - Colour-coded visual feedback.
- Expected backend behaviour:
    - update_mu_filter_button_pushed() extracts current MU parameters

Test 3: Extend MU filters

Description: Test the filter extension functionality that applies the current motor unit filter to the entire signal duration, extending beyond the current viewing window to identify additional motor unit activations.

Steps:

1. Click the ‘Extend MU Filter’ button
2. Monitor process progress across signal duration
3. Verify filter application to entire signal length
4. Check for newly identified spikes outside current view
5. Confirm SIL value recalculation after extension

Expected Results:

- Expected UI behaviour:
    - Extended filter button active when MU selected
    - Progress indicator shows processing stages
    - Pulse train plot updates with extended results
    - Colour feedback indicates SIL improvement/degradation
    - Success notification displays completion status

- Expected backend behaviour:
    - extend_mu_filter_button_pushed() applies filter across full signal
    - SIL recalculation reflects extended motor unit quality

Test 4: Batch Processing

Description: Test automated batch processing operations including outlier removal, filter update across all motor units and flagged Mu deletion.

Steps:

4.1 setup:

1. Decomposed EMG data should be loaded in via the decomposition tab.
2. Verify successful data loading with motor unit spike trains visible.

4.2 batch outlier removal:

1. Click “remove outliers” button
2. Wait for batch processing completion
3. Verify outlier spikes have been removed across all motor units
4. Check legitimate spikes remain intact
5. Confirm processing results are logged

4.3 batch filter update:

1. Click ‘MU filter update’ button
2. Set new filter parameters
3. Initiate batch filter update across all motor units
4. Check that filter changes affect spike detection appropriately
5. Confirm filter update results are saved

4.4 flagged MU deletion:

1. Select/flag motor units to delete and click ‘delete spikes’ button
2. Verify motor units are removed from the dataset
3. Confirm remaining motor units remain unaffected

Expected Results:

- Expected UI behaviour:
    - 4.1:
        - Data loads successfully with clear visual confirmation in the MU Editing tab
        - Motor unit spike trains displayed clearly with proper scaling and identification labels
        - Batch processing buttons and controls visible and enabled

        - Error messages display clearly if data loading fails
    - 4.2:
        - ‘Remove outliers’ button clearly visible and responsible when clicked
        - Spike train displays update after completion to show data with outliers removed
    - 4.3:
        - ‘Update MU filter’ button accessible and responds when clicked
        - Filter parameter dialog opens with clear input and parameter display
        - Motor unit displays refresh to show updated spike detection results
    - 4.4:
        - Motor unit selection interface intuitive with clear visual feedback for flagged units
        - Immediate visual update showing deleted motor units removed from display
- Expected backend behaviour:
    - Batch operation management through batch_processing.py
    - Filter update processing through mu_filter_actions.py
    - Outlier detection algorithm using machine learning algorithm via scikit-learn integration, ensuring accuracy
    - Errors are logged via logger.py

Tab 4: MU Analysis Tab

Test 1: MU Analysis functionalities

Description: test MU analysis functionalities including motor unit properties calculation, threshold analysis, force analysis and data visualisation capabilities.

Steps:

1.1 setup:

1. Load file containing data - this can be done by uploading a new file via ‘Press here to select file’ button in the MU Analysis tab or clicking the ‘next’ button from previous tabs.
    a. Once successfully uploaded/loaded, the data will show in the ‘select a file to begin the analysis’ section of the page.
2. Confirm all MU analysis sections are visible and accessible.

1.2 force analysis:

1. Access the force analysis dropdown section
2. Configure force analysis parameters
    a. Set force signal sampling rate to 2048 Hz
    b. Set time window
    c. Configure MVC detection sensitivity settings to 0.8

3. Calculate rate of force development (RFD) measurements
4. Review force analysis statistics and summary metrics

1.3 motor unit properties analysis:

1. Access the ‘Motor Unit Properties’ dropdown
2. Input required analysis parameters:
    a. Enter MVC (100)
    b. Set number of firings to 10 for analysis
3. Select motor units from the available dataset for analysis
    a. Check MU 1, 2 and 3
4. Click the calculate button
5. Review generated motor unit properties:
    a. Firing rate statistics
    b. Discharge rate patterns
    c. Motor unit activation thresholds
6. Verify results are displayed in ‘results to view’ section

1.4 data visualisation

1. Access the ‘plot EMG’ dropdown
2. Select visualisation options and parameters from available controls
    a. Plot type: select ‘Raw EMG’
    b. Select a short time range
    c. Channels: select channel 1-8
    d. Motor units: check MU 1 and 2
3. Generate EMG signal plots and verify proper visualisation rendering
4. Test plot interaction features (zoom, selection)
5. Verify multiple plot types are available (raw EMG, filtered signals, spike trains)
6. Check that plots update correctly when parameters are modified

Expected Results:

- Expected UI behaviour:
    - 1.1:
        - File upload button is clearly visible and responsive
        - Successful file loading indicated by file information displayed in the centre of page
        - All dropdown sections visible with expand/collapse functionality
        - File metadata correctly displayed after successful file upload
        - Error messages displayed for incorrect or invalid file format
    - 1.2:
        - Force analysis dropdown expands to reveal parameter controls and analysis options
        - MVC detection sensitivity controls responsive with real-time feedback

        - Force analysis results displayed in organised format in results section on the right side panel
    - 1.3:
        - Motor unit properties dropdown expands to reveal parameter inputs
        - Number of firings parameters clearly labeled and validated
        - Calculate button responsive with progress indication during computation
    - 1.4:
        - Plot EMG dropdown expands to reveal visualisation controls and options
        - Interactive plots generated with labelling
        - Multiple plot types available and selectable
- Expected backend behaviour:
    - File loading and parsing through FileUploadFunc with proper data validation and error handling
    - Force analysis implementation via ForceAnalysisFunc using openhdemg functions (get_mvc, compute_rfd) for accurate MVC and RFD calculations
    - Plot generation through PlotEMGFunc with integration to matplotlib and pyqtgraph for accurate and interactive visualisations
    - Data and results are stored in AnalysisResultsHist
    - Errors are logged in logger.py

Manual End-to-End Testing

Exporting and Loading Sessions

Description: testing the application from importing data through complete analysis workflow to final session export, verifying all major functionality works together and sessions can be exported reliably.

1.1 Setup:

Steps:

1. Run the application
2. Upload data/trial_20MVC.obt+ (or any compatible file) via the import data tab
    a. Click ‘press gere to select a file’ button’
3. Click ‘next’ button in the footer

Expected UI behaviour:

- Application window opens with Import Data Tab active
- After file selection, file info label shows the filename
- Preview plot displays mean hdemg signal amplitude graph
- Footer displays: filename, size, format
- Set configuration, segment session and channel viewer buttons are enabled
- Recent files panel on left sidebar updates with uploaded file
- ‘Next’ button becomes enabled

Expected backend behaviour:

- emg_obj.open_otb_plus() called, loads signal data into emg_obj.signal_dict
- Processed file saved into data/filename_processed.mat
- H5 readin file created: data/filename_readin.h5
- Database:
    - New session created in session table with sessionid
    - File inserted into files table with fileid, linked to sessionid
    - File version inserted into file_versions table: stageid, filepath and last_opened timestamp recorded

1.2 Decomposition workflow:

Steps:

1. Set algorithm to ‘Fast ICA’
2. Advanced options configurations:
    a. Contrast function: skew
    b. Initialisation: random
    c. Peel off: yes
    d. Refine motor units: yes
3. Parameters configurations:
    a. Iterations: 10
    b. Windows: 1
    c. Threshold target: 0.9
    d. Extended channel: 100
    e. Duplicate threshold: 0.3
    f. SIL threshold: 0.9
    g. CoV threshold: 0.5
4. Click ‘Start decomposition’ button
    a. Monitor status via the progress bar (expected to take 2-5 minutes)
5. Decomposition results should appear in recent files panel
6. Click the ‘next’ button in the footer

Expected UI behaviour:

- Data file should be loaded in from clicking the ‘next’ in the previous tab
- Algorithm dropdown shows ‘Fast ICA’ selected
- All parameters populated with specified values
- After clicking ‘start decomposition’, ‘stop decomposition’ should become clickable
- Progress bar shows status of decomposition
- Progress percentage goes from 0% to 100% once completed
- The decomposed data file should appear on the recent files panel on the left sidebar
- The ‘next’ button should be enabled

Expected backend behaviour:

- DecompositionApp.start_decomposition() triggered
- FastICA decomposition runs with specific parameters
- Output file saved as filename_decomp.mat
- H5 decomp file created: filename/_decomp.h5
- Database:
    - File_version inserted into file_versions
        - Stageid = 3, filepath = data/filename
    - Log entry created in log table

1.3 Manual editing workflow:

Steps:

1. Quick edits (for more in-depth behaviour testing, refer to the manual functional testing section):
    a. Remove outliers from MU 1:
        i. Select MU 1 from motor unit list
        ii. Click ‘remove outliers’ button
        iii. Verify outlier spikes removed (red markers disappear)
2. Click the ‘save’ button
    a. Output files created: filename_processed.mat_decomp.mat.pyedited.mat
    b. H5 file created: filanem_edited.h5
3. Click the ‘next’ button in the footer

Expected UI behaviour:

- Decomposed data loaded into the tab already
- Motor unit list (left panel) displays detected motor units
- Centre plot shows
- After selecting MU 1, plot focuses on MU 1 spike train
- After clicking ‘remove outliers’, red outlier markers disappear from plot
- After clicking ‘save’, recent files panel updates with new edited file entry
- The ‘next’ button in the footer is enabled

Expected backend behaviour:

- MUeditManual.import_data() loads decomposition results
- Spike data rendered in PyQtGraph plot widget
- Database:
    - File version inserted into file_versions:
        - Stageid = 4, filepath = data/filename_edited.h5
    - Log entry created in logs table

1.4 Manual analysis workflow:

Steps:

1. File shows in the middle section of the page
2. Quick analysis (for more in-depth behaviour testing, refer to the manual functional testing section):
    a. Expand ‘force analysis’ dropdown
    b. Set force sampling rate to 200
    c. Click ‘calculate RFD’

Expected UI behaviour:

- File should be loaded in from clicking ‘next’ in previous tab
- After clicking ‘Calculate RFD’, results table appears showing (after a couple of seconds):
    - Peak force
    - Time to peak
    - RFD
    - Average force

Expected backend behaviour:

- After clicking calculate RFD button, MUPropertiesFunc.calculate_frd() function called
    - Force derivative computed: dF/dt
    - Peak force identified: max(force_signal)
    - Time to peak: argmax(force_signal) / fsamp
    - RFD: peak_force / time_to_peak
- Results stored in AnalysisREsultsHist.store
- Database:
    - Log entry updated in logs table

<u>1.5 Session export:</u>

Steps:

1. Click the ‘Export Session’ button on the left sidebar
2. Save the file to desired location
3. Verify the files being zipped:
    a. Filename_readin.h5
    b. Filename_decomp.h5
    c. Filename_edited.h5

Expected UI behaviour:

- After clicking export session button:
    - File dialog open
    - User saves into file
- After successfully saving, files should appear in the saved location.
    - ZIP creation structure (example):
        - trial1_20MVC_session.zip
        - ├── trial1_20MVC_readin.h5

        - ├── trial1_20MVC_decomp.h5
        - ├── trial1_20MVC_edited.h5
        - ├── readin_config.json
        - ├── decomposed_config.json
        - └── edited_config.json

Expected backend behaviour:

- Database query:
    - get_session_files(self.sessionid) retrieves all file versions for current session
    - Returns list of file objects
- Logger records successfully saving and the path

1.6 Loading session:

Steps:

1. Close application
2. Reload application
3. Click the ‘load session’ button on the left sidebar
4. Select the file that was exported and saved in step 1.5
5. All the files from the previous session should appear in the recent files section on the left sidebar

Expected UI behaviour:

- Application reopens to fresh state
- After clicking ‘load session’, file dialog opens:
    - After clicking on the session that was just saved, all the files should appear in the recent file panel (left sidebar)

Expected backend behaviour:

- Database:
    - New session created: sessionid = create_new_session()
    - Files inserted with new session linkage
- Logger records successfully loading of a previously saved session